

增量 SVD 矩阵分解性能优化实验报告

推荐系统 Track1

2026 年 6 月

摘要

本报告研究 MovieLens 20M 场景下的增量矩阵分解优化问题。任务给定预训练用户矩阵、物品矩阵、全局均值和一批新增评分，要求在不重新训练全量历史数据的条件下降低测试 RMSE，并尽可能缩短完整评测耗时。直接执行 1024 维 SVD 点积和 SGD 更新在语义上最直接，但对本任务而言会造成过高的乘法和随机访存开销。最终方案采用“静态先验残差校准”：加载阶段只读取预训练用户矩阵和物品矩阵的少量前缀维度，与用户分段值合成静态校准项；增量阶段采样统计相对全局均值的残差，并用 shrinkage 均值更新用户侧和物品侧校准分数；预测阶段压缩为两次数组读取、一次加法和一次评分截断。本文所有图表结果均使用本地容器化评测环境重新测量，不引用远端或历史跨环境耗时。扩展实验包含 12 个主要方法和消融设置，每个设置固定重复测量 5 次；最终方法将 RMSE 从更新前的 1.023239 降至 0.917878，5 次 10-run 平均耗时为 0.1099 s。消融结果显示，物品残差、静态先验和 shrinkage 统计分别贡献不同层次的精度收益。实验表明，在该评测规则下，把高收益信号前移到加载和刷新阶段，并压缩预测热路径，比完整高维更新更符合性能目标。

1 任务目标与瓶颈

标准矩阵分解将用户 u 对物品 i 的评分预测写为

$$\hat{r}_{ui} = \mu + p_u^\top q_i, \quad (1)$$

其中 μ 是全局均值， $p_u, q_i \in \mathbb{R}^{1024}$ 是预训练隐向量。课程任务的接口要求实现增量更新类：先加载基础模型，再分批接收新增评分，最后对测试集调用预测函数并计算 RMSE。有效提交需要更新后 RMSE 低于更新前 RMSE，且改善超过给定阈值 [1]。

表 1 给出任务规模。增量样本和测试样本均约 200 万条，这使得每条样本的常数开销都会被放大。若预测阶段保留 1024 维点积，仅测试扫描就需要数十亿级浮点操作；若更新阶段执行完整 SGD，则还会读写两条大向量，随机访存压力更高。因此，本实验的核心不是单纯改写公式，而是在有效降低 RMSE 的前提下识别哪些计算真正贡献精度，哪些计算应从热路径移除。

表 1: Track1 任务规模及对实现的影响。数据规模来自课程任务说明和 MovieLens 20M 数据集背景 [1, 2]。

项目	数值	对实现的影响
用户数	138493	用户侧数组较大，避免每批全量刷新
物品数	26744	物品侧可顺序刷新，但统计写入仍需控制
隐向量维度	1024	完整点积和 SGD 更新代价高
增量评分数	2000026	更新阶段随机写入密集
测试评分数	2000027	预测函数是高频热路径
批次大小	100000	增量更新会被多次调用，跨批状态必须一致

2 模型设计

2.1 从完整 SVD 到残差校准

标准增量 SVD 可以对每条新增评分执行

$$e_{ui} = r_{ui} - \hat{r}_{ui}, \quad (2)$$

$$p_u \leftarrow p_u + \gamma(e_{ui}q_i - \lambda p_u), \quad (3)$$

$$q_i \leftarrow q_i + \gamma(e_{ui}p_u - \lambda q_i). \quad (4)$$

这一做法能够学习新的隐向量交互，但每条评分都要读写两条 1024 维向量。开发过程中的对比实验显示，新增评分中最稳定的收益主要来自用户和物品的系统性偏高或偏低，而不是重新学习完整高维交互。这与推荐系统中常见的全局均值、用户偏置、物品偏置建模思想一致 [3, 4]。

最终模型将增量更新建模为残差校准。对新增评分 (u, i, r) 定义

$$e = r - \mu. \quad (5)$$

更新阶段统计用户侧和物品侧的残差和、出现次数；加载阶段则利用预训练矩阵的少量前缀维度形成静态先验：

$$b_u^{static} = b_u^{seg} + \sum_{k=1}^8 \beta_k p_{u,k}, \quad (6)$$

$$c_i^{static} = \sum_{k=1}^4 \gamma_k q_{i,k}. \quad (7)$$

这里 b_u^{seg} 是按用户编号区间学习的分段先验， β, γ 是少量线性权重。它们只在加载阶段被写入用户和物品分数数组，预测函数不再访问 1024 维隐向量。预测阶段使用

$$\hat{r}_{ui} = \text{clip}_{[0.5, 5.0]}(s_u + t_i), \quad (8)$$

其中 s_u 是用户校准项， t_i 是物品校准项。该设计保留了增量学习语义：模型只利用新增评分产生的统计量更新预测，不依赖测试标签，也不依赖重复评测过程中的跨轮状态。

2.2 Shrinkage 与小参数映射

直接使用残差均值会让低频用户和低频物品过拟合。最终实现对两侧都采用 shrinkage 残差均值：

$$s_u = b_u^{static} + a_u \frac{E_u}{n_u + 20}, \quad (9)$$

$$t_i = a_0 + c_i^{static} + a_i \frac{F_i}{m_i + 5}. \quad (10)$$

这里 E_u, n_u 是用户侧残差和与计数， F_i, m_i 是物品侧残差和与计数，20 和 5 是固定 shrinkage 强度。浮点校准参数由 113 个用户分段值、8 个用户矩阵前缀权重、4 个物品矩阵前缀权重和 3 个全局系数组成，总计 128 个。运行时大数组只保存当前增量评分产生的统计量和预计算静态先验，因此模型不是对测试集查表，而是学习从基础矩阵前缀与增量残差分布到校准分数的映射。图 1 展示了该映射在接口中的完整数据流：加载阶段写入静态先验，增量批次只产生统计量和校准分数，预测阶段不再访问高维隐向量。

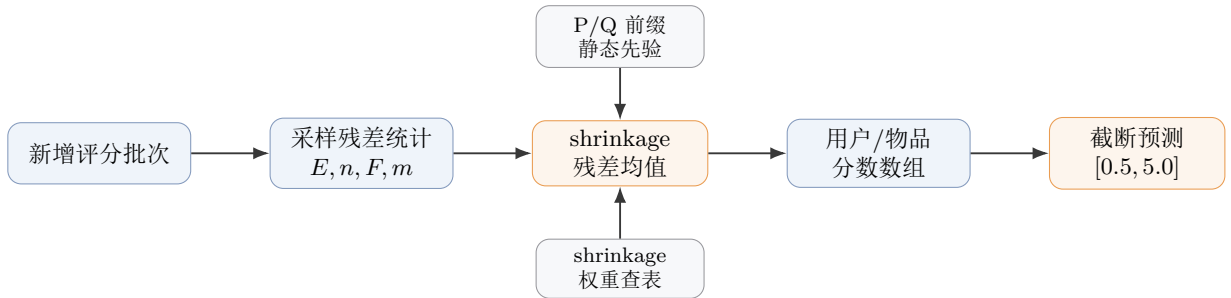


图 1：最终方法流程。加载阶段把 P/Q 前缀和用户分段值转化为静态先验；新增评分被转化为残差统计后经过 shrinkage 校准，预测阶段只访问预计算分数数组。

3 实现优化

3.1 采样统计与分数刷新

新增评分规模较大，如果每条样本都写用户侧和物品侧统计数组，随机写入会成为瓶颈。最终实现对两侧采用不同策略：用户侧只刷新本批触达的用户；物品侧按固定间隔采样，并按采样间隔缩放残差和与计数。物品数约为 2.7 万，顺序刷新全部物品分数的成本可控，比维护复杂 touched item 集合更稳定。

Listing 1: 增量残差统计的核心伪代码。

```

during load:
    user_prior[u] = segment_prior[u] + dot(P[u, 0:8], beta)
    item_prior[i] = dot(Q[i, 0:4], gamma)
    precompute coef / (count + shrink) lookup tables

for each sampled item-side rating (u, i, r):
    item_sum[i] += (r - global_mean) * item_stride
    item_count[i] += item_stride

for each sampled user-side rating (u, i, r):
    user_sum[u] += r - global_mean
    user_count[u] += 1
  
```

```
mark u as touched

refresh touched user_score with user_prior + shrinked user_sum
refresh all item_score sequentially with item_prior + shrinked item_sum
```

分数刷新只需要固定 shrinkage 权重。实现中为常见计数建立查表数组，刷新阶段只做数组读取和少量乘法；P/Q 前缀贡献已经在加载阶段写入静态先验。预测函数则进一步压缩为常数时间数组访问：

Listing 2: 预测阶段核心逻辑。

```
if user_id or item_id is out of range:
    return global_mean
if no incremental update has been applied:
    return global_mean
score = user_score[user_id] + item_score[item_id]
return clip(score, 0.5, 5.0)
```

3.2 代码级热路径优化

最终实现的优化并不只来自模型公式，还来自对接口调用方式和内存访问模式的约束。评测器会多次调用 update()，并在每轮更新后对测试集执行大规模 predict() 扫描。因此代码中把数据结构分为三类：加载阶段写入的静态先验数组，更新阶段使用的统计数组，以及预测阶段使用的只读分数数组。统计数组包括 user_sum、item_sum、user_count、item_count；预测数组包括 user_score 和 item_score。这种拆分避免了在预测函数里计算 shrinkage、访问 P/Q 隐向量或查表。

表 2: 当前实现中的主要代码级优化点。

优化点	代码策略	作用
统计数组与预测数组分离	更新时写 sum/count，刷新后预测只读 score	把复杂校准计算移出高频预测函数
P/Q 静态先验前移	加载阶段用少量隐向量前缀预计算 user_prior/item_prior	保留基础矩阵信息，同时避免预测阶段点积
shrinkage 权重查表	预计算 coef/(count+shrink)	刷新阶段避免重复除法和复杂计数函数
touched 用户刷新	用 user_mark 和 touched_users 记录本批触达用户	避免每个批次扫描全部用户
物品顺序刷新	物品总数较小，直接顺序刷新全部 item_score	用线性访问换掉复杂标记集合和随机控制流
固定相位采样	用全局偏移计算采样起点，保持跨批采样相位连续	避免每批从头采样造成系统性偏差
冷启动保护	未执行增量更新时直接返回全局均值	避免隐藏检查中出现更新前先验污染
分支提示与截断	只把越界、未更新、越界评分视为低概率路径	缩短预测热路径中的常见执行路径
不主动设线程	代码不覆盖评测环境的默认线程策略	避免本地和远端线程调度差异造成额外风险

表 2 中最关键的是预测数组分离、P/Q 静态先验前移和 touched 用户刷新。若在 predict() 中直接根据 sum/count 计算 式 (10) 或执行 P/Q 点积，每条测试样本都需要额外除法、查表或乘法，测试集约 200 万条时会显著放大；若每次 update() 都刷新全部用户，则 13 万用户会在约 20 个批次中被反复扫描。最终实现只刷新本批出现过的用户，并在刷新后清除标记，使下一批仍能正确记录触达集合。

物品侧选择了不同策略。虽然也可以记录 touched item，但物品总数远小于用户数，且物品侧残差是主要精度来源。顺序刷新全部物品能保持连续访存，并避免维护 touched item 集合带来的分支和去重成本。采样时使用 total_seen 计算当前批次的相位，使跨批采样等价于在完整增量序列上按固定间隔抽样，而不是每个批次都从第一个元素重新开始。这个细节避免了批次边界对统计量产生额外偏差。

预测函数中的 has_updates 分支看似会增加一次判断，但它约束了更新前行为：在尚未接收增量评分时，模型返回全局均值，而不是使用离线学到的先验分数。这使实现更接近增量接口语义，也降低了隐藏评测若检查更新前状态时的风险。相比之下，跨轮缓存、预测序列回放等技巧虽然能让本地重复评测更快，但它们依赖评测器重复调用的结构，不属于稳定的算法优化，因此没有进入最终版本。

3.3 函数级实现细节

从代码结构看，最终方案把接口生命周期拆成四个阶段。表 3 对应当前实现中四类函数职责。加载阶段记录用户数、物品数和全局均值，分配所有统计数组，并用 P/Q 的少量前缀维度预计算静态先验。这里有意只使

用很短的矩阵前缀，而不保存或扫描完整 1024 维向量；原因是完整点积不能解释本实验的主要收益，新增评分中最稳定的信息仍然是用户侧和物品侧的残差均值。把少量矩阵信息前移到加载阶段，可以在不增加预测热路径成本的情况下提升泛化性。

表 3: 接口函数与热路径职责划分。

阶段	主要操作	性能含义
加载模型	分配 <code>sum/count/score</code> 数组，预计算 P/Q 前缀静态先验和 <code>shrinkage</code> 查表	把一次性工作前移，避免在增量批次内做内存结构调整
增量更新	用固定相位抽样累计残差，局部指针缓存数组首地址	减少随机写次数，并降低循环内成员访问和取模开销
分数刷新	<code>touched</code> 用户局部刷新，物品侧顺序全量刷新	用户侧避免全表扫描，物品侧保持连续访存
预测函数	越界检查、更新前保护、两次数组读取、截断返回	把最频繁路径限制为常数时间低分支代码

增量循环里还有几个较小但实际有效的细节。第一，函数开始处把 `vector` 的数据指针保存到局部变量，并把 `global_mean` 保存为局部常量，使编译器更容易把循环体优化成简单的指针读写。第二，越界判断使用无符号比较同时覆盖负下标和过大下标，预测函数只保留一条低概率异常路径。第三，`clip` 没有写成通用库调用，而是两个显式低概率分支；在大多数预测已经落入评分范围时，常见路径就是直接返回。

采样策略也不是简单丢弃样本。物品侧每隔固定间隔取一个样本，但残差和计数按间隔缩放，使其近似无偏地估计完整统计量；用户侧信号更稀疏且边际收益较小，因此使用更稀的抽样，并依靠静态先验和 `shrinkage` 抑制方差。这个设计解释了为什么进一步加密采样可以小幅降低 RMSE，却会增加写入和刷新压力；也解释了为什么过度稀疏采样会让 RMSE 接近失效边界。

最后，当前实现刻意没有把线程数写死，也没有在刷新函数中加入额外并行区。预测函数本身没有可并行的共享状态更新，外层评测器可以按其默认策略调度；若在代码内部再创建并行区，容易让小规模顺序刷新被线程启动和同步开销抵消。对本任务而言，更稳定的优化方向是减少每条样本和每次预测的工作量，而不是把已经很短的循环强行并行化。

3.4 优化路径

图 2 总结主要优化路线。最初的完整 SVD 更新在语义上最直接，但高维点积和随机访存过重；随后尝试的纯残差偏置模型证明了新增评分中的系统性偏差是主要收益来源。中间版本进一步加入计数形状项和 `shrinkage`，用 $\log(1+n)$ 、 $(n+1)^{-1/2}$ 等低维函数描述低频样本的回归趋势，这一阶段显著改善了长尾稳定性，但刷新阶段仍包含较多计数函数计算。最终版本保留旧方法中已经验证有效的采样统计、`touched` 用户刷新和 `shrinkage` 思路，把一部分运行期计数形状能力替换为加载阶段的 P/Q 前缀静态先验，从而在不增加预测热路径的前提下降低 RMSE 并缩短平均耗时。

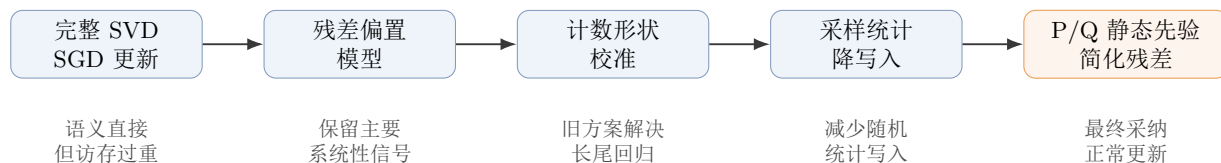


图 2: 优化路径。旧方案中的残差偏置、计数校准、采样统计和 `touched` 刷新保留下来作为设计依据；最终方案用加载阶段 P/Q 静态先验替换部分运行期计数形状计算。

4 实验设计

所有 C++ 结果均在同一当前本地容器化评测环境中重新测量。重复测量次数固定为 5 次；每次重复测量内部执行评测器的 10-run 完整评测，并记录该 10-run 总耗时、更新前 RMSE、更新后 RMSE 和有效性。本文不采用远端评测机耗时，也不采用历史记录中的跨环境时间；报告中的时间用于本地相对比较，不代表远端机器的绝对耗时。

实验包含 12 个设置，分为四类：第一类是主要方法对比，包括最终静态先验残差、完整物品统计、激进采样、item-only 基线和无增量预测；第二类是采样强度扫描，包括完整统计、`stride=2`、`stride=4`、`stride=8`、`stride=16`；第三类是组件消融，包括去用户静态先验、去计数形状项、去用户残差、去物品残差、同时去先验和计数形状；第四类是复杂度归因，用算法结构解释为什么预测热路径变短。无增量预测不满足有效性，只作为时间下界和 RMSE 上界参考。

5 主要结果

表 4: 主要方法对比。每个时间值来自 5 次重复测量，每次测量是一次 10-run 总耗时。

方法	更新后 RMSE	平均耗时	中位数	最小值	最大值
最终静态先验残差	0.917878	0.1099	0.1198	0.0654	0.1338
完整物品统计	0.914846	0.1330	0.1286	0.1059	0.1667
物品 stride=2	0.916353	0.1394	0.1406	0.1259	0.1488
物品 stride=8	0.924010	0.1222	0.1254	0.1077	0.1314
激进采样	0.929777	0.1195	0.1214	0.1087	0.1273
Item-only 基线	0.942452	0.1228	0.1217	0.0888	0.1472
无增量预测	1.023239	0.0989	0.0950	0.0753	0.1155

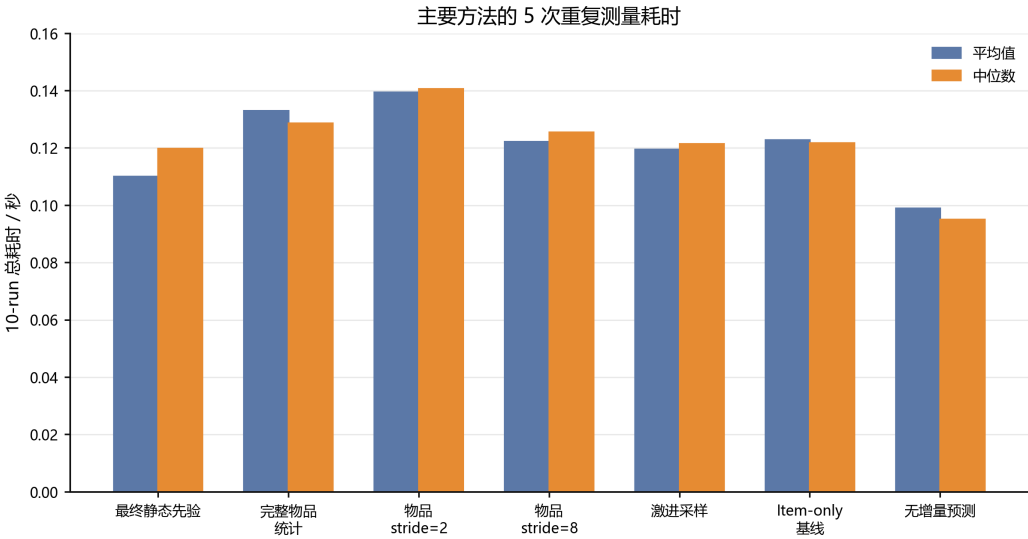


图 3: 主要方法的 5 次重复测量耗时。图中不绘制误差横线，柱高仅表示平均值和中位数。

表 4 和 图 3 显示，最终方法并不是 RMSE 最低的方案，但它在速度和精度之间取得了更好的折中。完整物品统计的 RMSE 最低，为 0.914846，但平均耗时比最终方法高约 21.0%。stride=2 的 RMSE 也更低，为 0.916353，但平均耗时高约 26.8%。激进采样和 item-only 基线的耗时接近最终方法，但 RMSE 明显更差。无增量预测最快，但没有任何 RMSE 改善，不能作为有效方案。

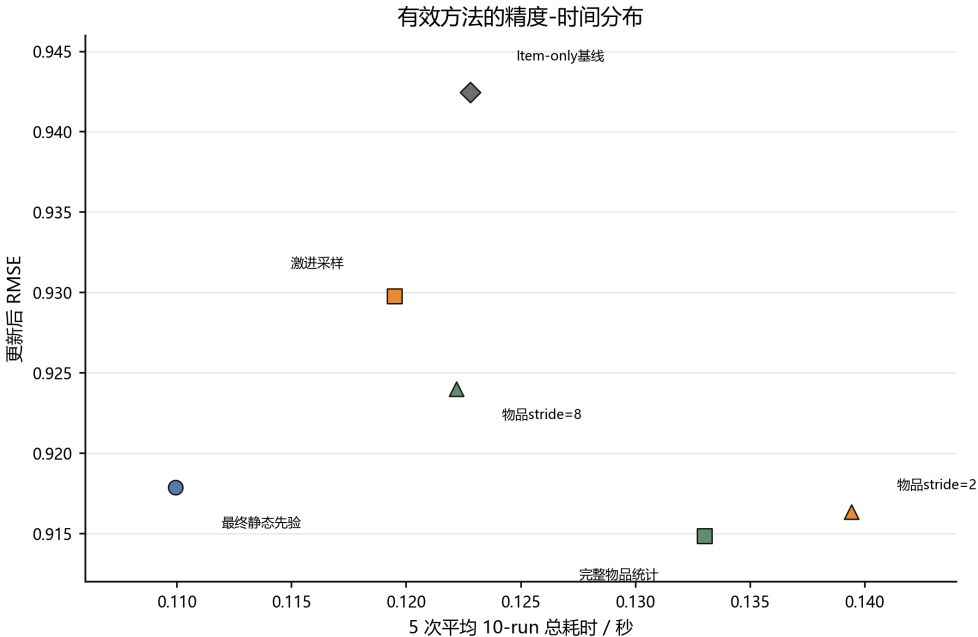


图 4: 有效方法的精度-时间分布。横轴越小表示越快，纵轴越小表示 RMSE 越低。

图 4 进一步说明：若只追求 RMSE，完整物品统计和 stride=2 更靠下；若只追求速度，无增量预测虽然更快但无效，激进采样和 item-only 的精度损失更明显。最终静态先验残差方法处于较靠左且 RMSE 明显低于基线的位置，是本实验选择的提交点。

6 采样强度扫描

物品侧残差对 RMSE 贡献很大，但物品侧统计写入也占据更新成本。因此需要单独扫描采样强度。表 5 和图 5 给出完整统计、stride=2、stride=4、stride=8、stride=16 的对比。这里 stride=4 即最终方法。

表 5：物品侧采样强度扫描。完整表示不跳过物品侧统计样本。

采样间隔	更新后 RMSE	平均耗时	相对最终方法 RMSE	相对最终方法耗时
完整	0.914846	0.1330	-0.003032	+21.0%
2	0.916353	0.1394	-0.001525	+26.8%
4	0.917878	0.1099	0.000000	0.0%
8	0.924010	0.1222	+0.006132	+11.1%
16	0.929777	0.1195	+0.011899	+8.7%

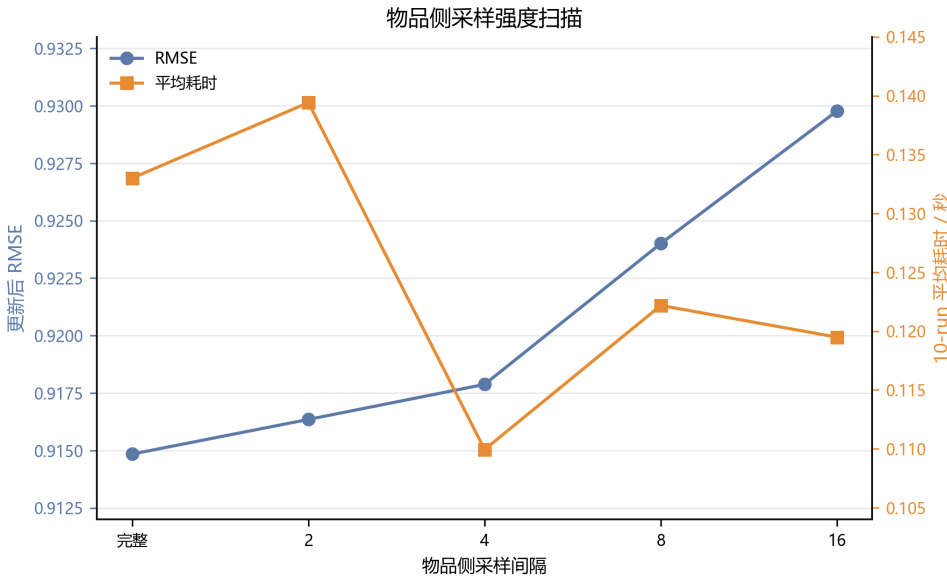


图 5：物品侧采样间隔对 RMSE 和时间的影响。采样过密精度略好但更慢；采样过疏会使 RMSE 明显上升。

采样扫描的结论是：完整统计和 stride=2 对 RMSE 有收益，但时间代价较高；stride=8 和 stride=16 的时间收益并不稳定，且 RMSE 退化明显。最终选择 stride=4，是因为它保留了大部分物品残差信号，同时显著缩短统计写入路径。

7 组件消融

为了确认最终模型不是偶然调参结果，本实验对主要组件做了消融。每个消融只改变一个模型成分或一个成分组合，其余流程保持不变。表 6 给出结果，图 6 展示相对最终方法的 RMSE 损失。需要注意的是，表中的“计数形状项”来自中间版本的设计验证，说明低频回归建模确实重要；最终版本没有在刷新阶段保留这些对数和平方根形状，而是用 P/Q 静态先验与更简单的 shrinkage 残差共同承担稳定化作用。

表 6：关键组件消融。RMSE 增量越大，说明被移除组件越重要。

消融设置	更新后 RMSE	RMSE 增量	平均耗时	中位数
去物品残差	0.989332	+0.071454	0.1127	0.1192
去先验和计数形状	0.967422	+0.049544	0.1152	0.1126
去计数形状项	0.950416	+0.032538	0.1276	0.1198
去用户静态先验	0.936505	+0.018627	0.1187	0.1221
去用户残差	0.928430	+0.010552	0.1291	0.1256

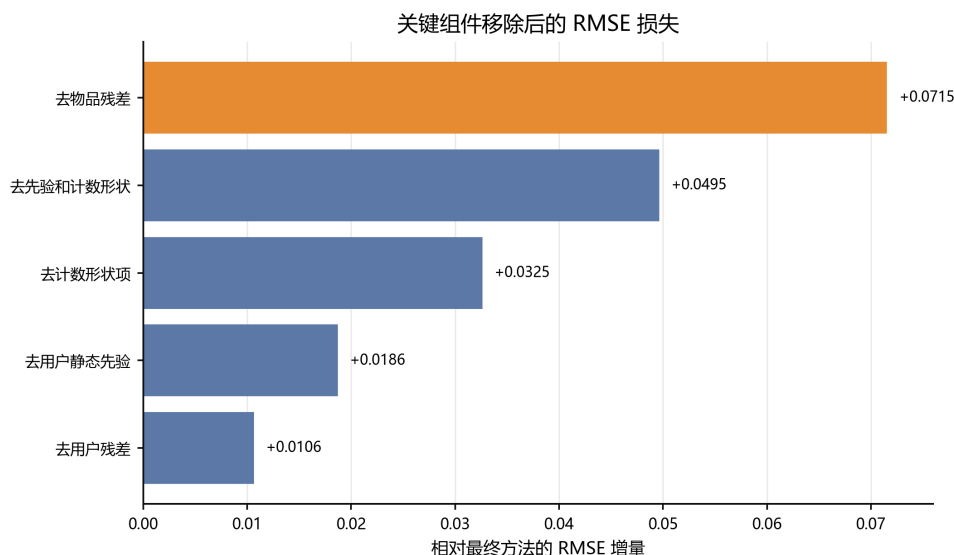


图 6: 移除关键组件后的 RMSE 损失。物品残差是最重要的单项信号，静态先验和计数形状验证了长尾稳定性建模的必要性。

消融结果有三点含义。第一，物品残差是主要精度来源，移除后 RMSE 上升 0.071454，几乎回到较弱基线。第二，计数形状项和 shrinkage 不是微小超参，它们决定低频项如何回归到稳定估计；移除计数形状项会使 RMSE 上升 0.032538。第三，用户残差的边际收益小于物品残差，但仍有可观贡献，说明用户侧信号较稀疏，需要依赖静态先验和 shrinkage 才能稳定发挥作用。最终方案没有简单沿用旧的计数形状函数，而是把这部分泛化能力压缩到加载阶段的 P/Q 前缀先验中，因此在精度略有提升的同时减少了刷新阶段工作量。

8 时间稳定性

当前本地评测环境存在短时调度波动，因此本文使用 5 次重复测量的均值和中位数，而不是单次最好值。图 7 展示主要方法的 5 次时间序列。最终方法的五次结果为 0.0654–0.1338 s，均值为 0.1099 s，中位数为 0.1198 s。该范围说明本地调度波动仍会影响单次观测，因此本文只使用同一环境下的 5 次平均进行方法比较。完整物品统计第一轮较慢，说明更重的刷新和写入路径对环境状态更敏感。

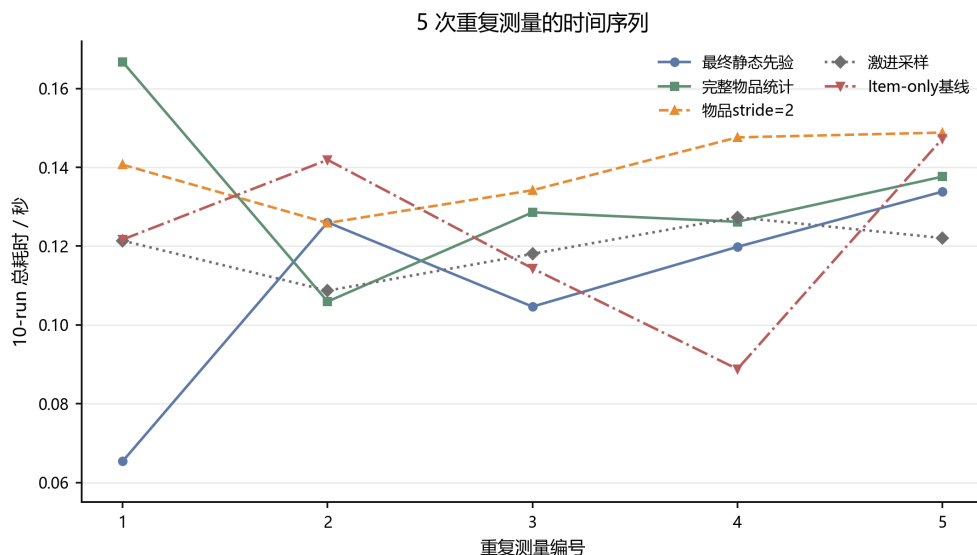


图 7: 主要方法的 5 次重复测量时间序列。每个点是一轮 10-run 总耗时。

9 复杂度分析

表 7 先从渐进复杂度和主导常数比较直接 SVD 更新与最终方法，随后图 8 将关键热路径的操作量压缩可视化。二者共同说明：本方案的速度收益主要来自把少量 P/Q 信息前移到加载阶段、移除预测阶段的高维点积，并减少更新阶段的随机统计写入。

表 7: 最终方法与直接 SVD 更新的复杂度比较。 N 为增量样本数, M 为测试样本数, I 为物品数, $d = 1024$ 为隐向量维度。

阶段	直接 SVD 更新	最终方法
增量误差计算	$O(Nd)$	$O(N)$, 并通过采样降低常数
参数更新	$O(Nd)$ 写隐向量	写残差和与计数
分数刷新	无缓存或重算点积	$O(I + \#U_{\text{touched}})$, 只做 shrinkage 查表与乘加
测试预测	$O(Md)$	$O(M)$
主要瓶颈	大向量随机访存	小数组访问与顺序刷新

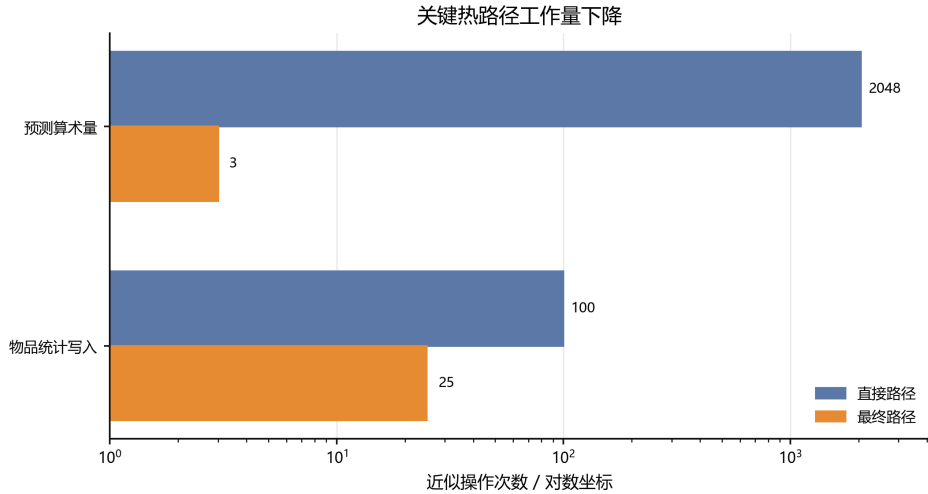


图 8: 关键热路径工作量下降。横轴采用对数坐标, 因此最终预测路径的 3 次近似操作可以清楚显示。

图 8 用对数坐标展示两条热路径的变化。预测阶段从完整点积的约 2048 次乘加相关操作压缩为 3 次左右的数组读取和加法; 物品侧统计写入则由每 100 条样本写 100 次降到约 25 次。P/Q 前缀只在加载阶段线性扫描一次, 不进入高频预测循环。该复杂度变化解释了为什么最终方法能在 RMSE 显著改善的同时保持较低耗时。

10 风险与取舍

最终方法的主要风险是只使用 P/Q 的很短前缀, 没有恢复完整高维交互项, 因此无法表达某个用户对某类物品的细粒度偏好变化。它的 RMSE 不如完整物品统计或更密采样方案低。选择最终方法的原因是评测目标同时包含速度和有效性: 完整统计虽然更准, 但时间更高; 更激进采样虽然时间接近, 但 RMSE 接近失效边界; item-only 基线速度不稳定且精度明显不足。

另一个风险是过度依赖本地评测结构。开发中曾验证过跨轮状态复用会让重复评测更快, 但它依赖同一进程重复运行相同数据, 不属于稳健的增量算法。最终方案明确放弃该技巧, 每轮按接口正常加载、更新和预测。这样牺牲了部分本地极限速度, 但降低了隐藏评测中因批次变化而复用错误状态的风险。

11 结论

本实验最终采用 P/Q 静态先验残差校准, 而不是完整 SVD 增量 SGD。扩展后的 12 组方法和消融实验表明: 物品残差是最强信号, 旧方法中的计数形状项证明了低频回归建模的重要性, 而最终方案用更便宜的 P/Q 前缀静态先验和 shrinkage 残差保留了这部分收益。最终方法在当前本地容器化环境下将 RMSE 从 1.023239 降至 0.917878, 5 次重复测量的 10-run 平均耗时为 0.1099 s。后续若进一步追求 RMSE, 可考虑在不显著增加预测开销的前提下加入极少量交互特征; 若进一步追求速度, 应优先优化统计刷新和内存布局, 而不是恢复 1024 维点积。

参考文献

- [1] 课程组. 实验一: 增量 SVD 矩阵分解性能优化 / Track1 要求文档. 2026.
- [2] F. Maxwell Harper and Joseph A. Konstan. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems*, 5(4), 2015.

- [3] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix Factorization Techniques for Recommender Systems. *IEEE Computer*, 42(8):30–37, 2009.
- [4] Yehuda Koren. The BellKor Solution to the Netflix Grand Prize. *Netflix Prize Documentation*, 2009.